



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

TensorRT SDK NVIDIA

Sección 1 — Fundamentos y Arquitectura

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Tabla de Contenidos

1 TensorRT

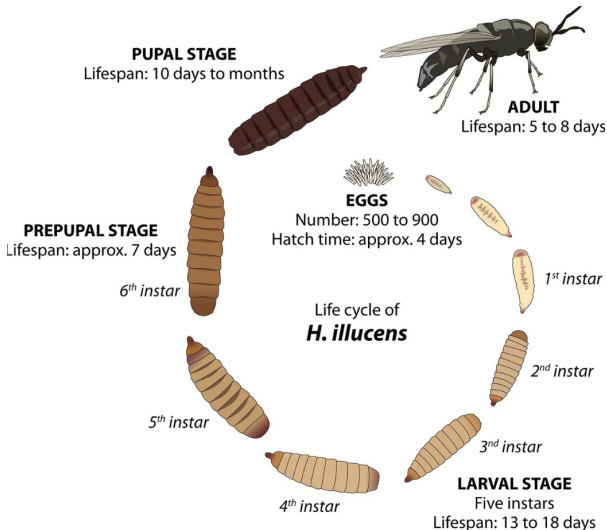
- Flujo de diseño de TensorRT
- Building phase
- Running Phase

2 GPU Architectures

3 Performance Assessment

- Latency
- Throughput
- Size of model

4 Ejemplo de aplicación



*"Many insects have a larval form that is optimized for extracting energy and nutrients from the environment and a completely different adult form that is optimized for the very different requirements of traveling and reproduction."*¹

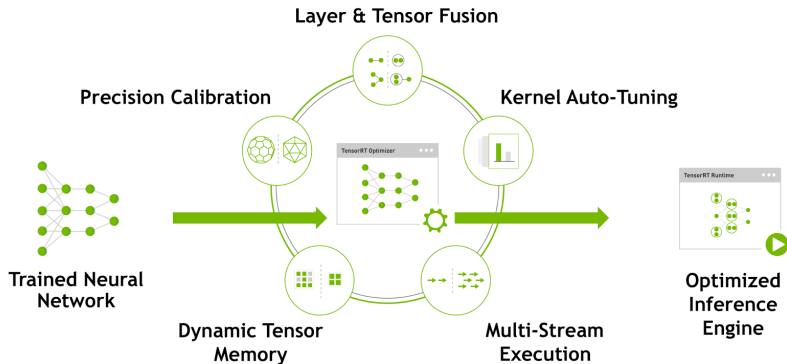
¹G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network", arXiv preprint arXiv:1503.02531, 2015.

TensorRT

Un inference optimizer es una herramienta o conjunto de técnicas utilizadas para mejorar la eficiencia y el rendimiento de un modelo de *machine learning* durante la fase de inferencia.

- ▶ Una vez que el modelo ha sido entrenado, puede ser procesado por un **inference optimizer**:
 - Compresión del modelo y mapeo de operaciones al hardware disponible.
 - Se genera el **inference engine** → información de como ejecutar operaciones del modelo ML en el hardware objetivo.

Inference Optimizer

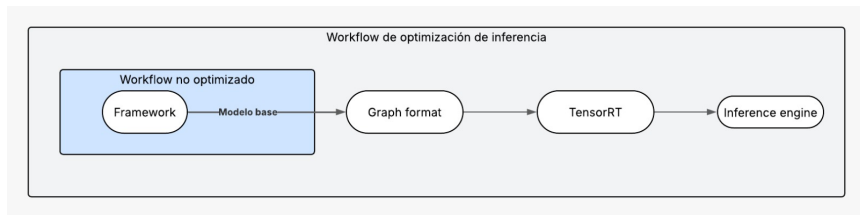


Dynamic tensor memory es una técnica que reutiliza y gestiona dinámicamente los buffers de tensores durante la inferencia para reducir el consumo total de memoria GPU.

Los *buffer* de tensores es la porción de memoria reservada para guardar los datos de un tensor mientras se procesan en la GPU.

Workflow no optimizado: no considera las particularidades de la arquitectura de *Deep Learning*, siendo agnóstico a ella.

No optimizado para una arquitectura específica, lo que implica desconocimiento del hardware subyacente y de sus capacidades.



²Juan Carlos Aguilera. UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA DEPARTAMENTO DE ELECTRÓNICA EXPLORACIÓN DE METODOLOGÍAS PARA LA OPTIMIZACIÓN DE INFERENCIA DE REDES NEURONALES EN GPU UTILIZANDO TENSORRT. Master's thesis, Universidad Técnica Federico Santa María, 2024

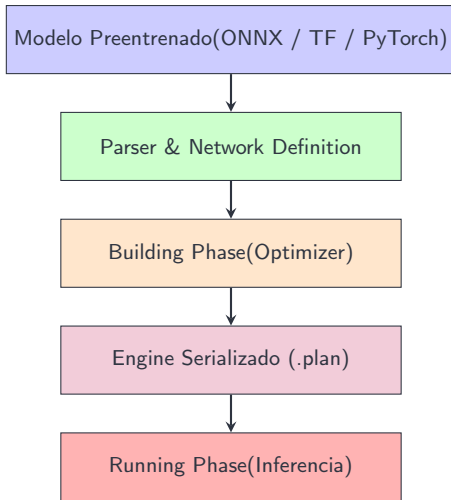
Algunos ejemplos de flujos de trabajo de optimización de inferencia reportados en la literatura reciente:

- ▶ **PyTorch - TensorRT:** Describe el proceso proporcionado por PyTorch para optimizar modelos utilizando TensorRT.
- ▶ **TensorFlow - TensorRT:** Aquí, el formato de grafo también está integrado dentro de TensorFlow.
- ▶ **PyTorch - ONNX - TensorRT:** Detalla la transición de un modelo al formato ONNX, que luego se optimiza utilizando TensorRT.

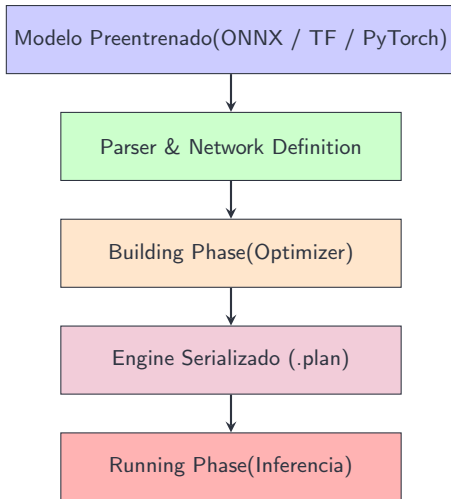
- ▶ TensorRT es un *Software Development Kit* (SDK) para generar *inference engines* optimizados.
- ▶ Diseñado para inference a partir de modelos preentrenados.
- ▶ Optimizaciones basadas en propiedades específicas del hardware de Nvidia.
- ▶ Es software cerrado, y los detalles internos no son públicos.
- ▶ La documentación oficial de Nvidia y estudios académicos ofrecen descripciones generales de sus optimizaciones.
- ▶ Se integra con frameworks populares de machine learning.
- ▶ Existe evidencia empírica sustancial sobre los *inference engines* generados por TensorRT.

- ▶ TensorRT proporciona APIs para C++ y Python.
- ▶ Permiten programar una **TensorRT Application**, que es una implementación de un inference optimizer mediante las funciones proporcionadas por TensorRT.
- ▶ Una TensorRT Application puede optimizar modelos y realizar inferencias.
- ▶ Los pasos para implementar una TensorRT Application se describen en la documentación oficial de TensorRT, pero no de manera exhaustiva.
- ▶ Los pasos específicos pueden variar según la aplicación, el nivel de optimización, el framework utilizado, etc.

Flujo de Diseño de TensorRT



- El flujo de TensorRT Application comienza con un **modelo base**.
- Sigue una **building phase** donde se optimiza la red para una GPU específica.
- Luego, en la **running phase**, se utiliza el modelo optimizado para ejecutar inferencias.



- Hay documentación y ejemplos en la web a fin de facilitar la adopción y el uso de la herramienta, abarcando desde decisiones de diseño hasta la reproducibilidad de resultados.

Flujo de diseño de TensorRT

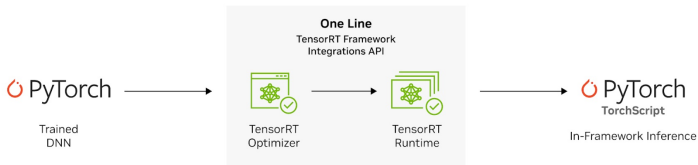
Pipeline de tres etapas principales:

1. Building phase.
2. Serialización del Engine³
3. Fase de Ejecución (Running Phase)

TensorRT

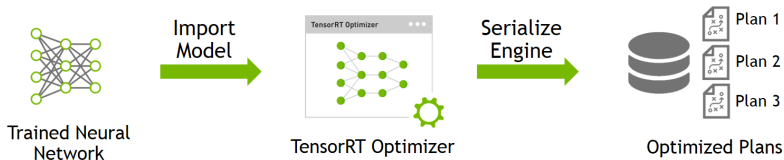


TensorRT Framework Integrations



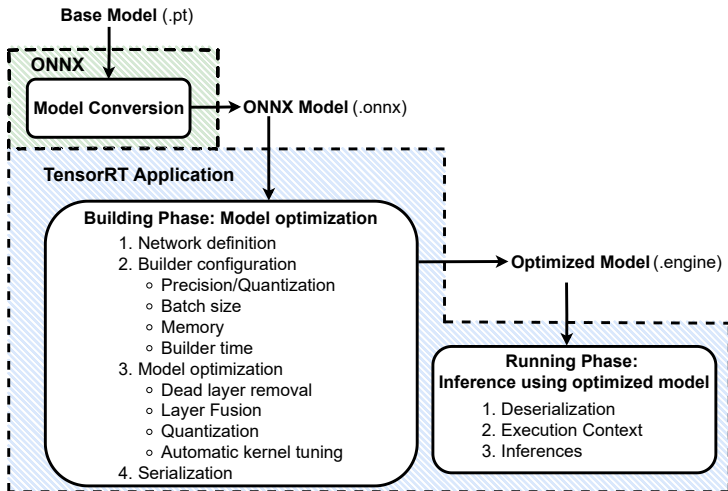
³Guardar la representación optimizada (engine) en un archivo binario, que luego puede cargarse para hacer inferencias sin necesidad de recompilar u optimizar de nuevo.

Building phase



Building phase

En el **building phase**, TensorRT desempeña el papel de builder, encargado de optimizar la red y producir un *inference engine*.



Network definition

- ▶ La **building phase** comienza con la generación de una **network definition**.
- ▶ Es una representación interna exclusiva de TensorRT.
- ▶ Incluye información detallada sobre:
 - Tipos de capas (convolucionales, *fully-connected*, activación, etc.).
 - Parámetros de cada capa.
 - Definición de entradas y salidas de la red con sus dimensiones.
- ▶ Los tensores no marcados como entradas o salidas se consideran valores transitorios que el builder puede optimizar.

Network definition

- ▶ Se puede obtener automáticamente mediante el **parser integrado en TensorRT⁴**.
- ▶ El parser es compatible con el formato de grafo **ONNX**.
- ▶ Alternativamente, TensorRT permite generar una network definition de forma manual utilizando funciones como `layer` y `tensor`.
- ▶ **El proceso manual está escasamente documentado y puede ser molesto.**
- ▶ El método más común es a través del uso de **ONNX**.
- ▶ Luego de obtener la network definition, es necesario describir la **builder configuration**.

⁴`parser` es un módulo que lee el grafo del modelo (por ejemplo, ONNX) y lo convierte automáticamente en una "network definition"

- ▶ **Previo a la optimización**, es necesario crear una configuración del builder.
- ▶ Esta configuración se puede modificar para personalizar la optimización del modelo.
- ▶ El ajuste de la configuración puede afectar el uso de recursos, la calidad de los resultados y la viabilidad del proceso de optimización.
- ▶ Los parámetros configurables están enumerados en la [documentación online](#).

Precision/quantization

- ▶ Controla cómo se representan los valores numéricos de los parámetros del modelo.
- ▶ Mayor precisión (fp32):
 - Garantiza cálculos más exactos.
 - Aumenta el consumo de recursos computacionales.
- ▶ TensorRT permite reducir la precisión a fp16 o cuantizarlos a int8.
- ▶ La cuantización puede mejorar la eficiencia a costa de una posible pérdida de precisión de la inferencia.

Batch size

- ▶ Define la cantidad de muestras que se procesan juntas en una sola iteración.
- ▶ Un batch size mayor puede incrementar la cantidad de inferencias realizadas por unidad de tiempo, pero también requiere más memoria.
- ▶ Se puede configurar un **batch size estático**:
 - La optimización del modelo se realizará exclusivamente para ese tamaño fijo.
 - El inference engine resultante solo podrá operar con dicho tamaño al ejecutar inferencias.
- ▶ Alternativamente, se puede optar por un **batch size dinámico**⁵:
 - Permite al inference engine realizar inferencias con una cantidad variable de muestras de entrada dentro de un rango definido en la configuración⁶.

⁵Batch size de configuración: Indica si el inference engine fue configurado con un batch size estático o dinámico.

⁶La cantidad de muestras de entrada que el modelo procesa durante una inferencia.

Memory

- ▶ Define el límite de memoria que cada capa del modelo puede utilizar durante el proceso de optimización.
- ▶ Se crean múltiples instancias del modelo, lo que incrementa los requerimientos de memoria.
- ▶ Se establece como la memoria máxima disponible en el dispositivo por defecto.
- ▶ Este parámetro aplica únicamente durante el diseño y generación del *engine*, no durante la inferencia.

Builder optimization level

- ▶ Define el nivel de optimización que el builder empleará para buscar y aplicar mejoras al modelo⁷.
- ▶ El comportamiento del builder según el nivel seleccionado:
 - **Nivel 0:** Habilita la compilación más rápida al deshabilitar la generación dinámica de kernels y seleccionar la primera táctica que se ejecute con éxito. No utiliza caché de tiempos.
 - **Nivel 1:** Ordena las tácticas disponibles mediante heurísticas y prueba únicamente las más prometedoras para seleccionar la óptima⁸.
 - **Nivel 2:** Ordena las tácticas disponibles por heurísticas, pero prueba únicamente las tácticas más rápidas para determinar la mejor.
 - **Nivel 3:** Aplica heurísticas para decidir si se utiliza un kernel precompilado estático o si es necesario compilar uno dinámicamente⁹.
 - **Nivel 4:** Siempre compila un kernel dinámico.
 - **Nivel 5:** Compila un kernel dinámico y lo compara con kernels estáticos.
- ▶ Un nivel más alto permite al *builder* probar una mayor variedad de combinaciones y tipos de *kernels*, a cambio de un mayor tiempo de optimización.

⁷El valor del parámetro es un número entero en el intervalo de 0 a 5.

⁸Si se genera un kernel dinámico, su nivel de optimización es bajo.

⁹kernel dinámico es un kernel que no está precompilado en una biblioteca estática, sino que se genera o ajusta en tiempo de optimización (*build time*) para adaptarse mejor a las características del modelo, el tamaño del *batch*, o el *hardware* específico.

Iterations

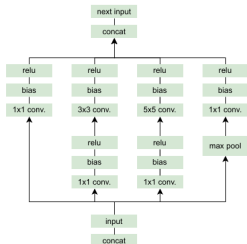
Define la cantidad de veces que se ejecuta cada táctica or kernel configuration durante la etapa of *automatic kernel tuning*.

- ▶ Por defecto, el *builder* evalúa el tiempo de ejecución de cada táctica una sola vez.
- ▶ Al incrementar el valor de *iterations*, se calcula un tiempo promedio, lo cual permite seleccionar la táctica más estable y rápida en promedio.
- ▶ Un mayor número de iteraciones reduce el impacto del ruido del sistema (por ejemplo, cambios en la frecuencia de la GPU o procesos en segundo plano).
- ▶ Ayuda a mejorar la convergencia del *inference engine*, disminuyendo el efecto del comportamiento no determinístico del sistema.

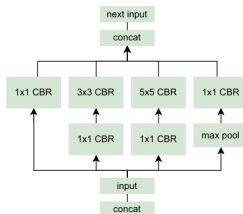
► El builder realiza optimizaciones como:

- Eliminar cálculos redundantes.
- Combinar y reordenar operaciones.
- Opcionalmente, reducir la precisión de los parámetros (fp32 a fp16 o cuantización de fp32 a int8).
- Evaluar el tiempo de ejecución de múltiples implementaciones para cada capa.

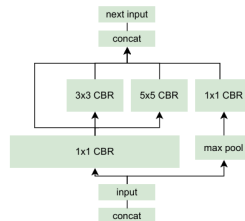
Model optimization



(a) Red neuronal.



(b) Fusión vertical de capas.



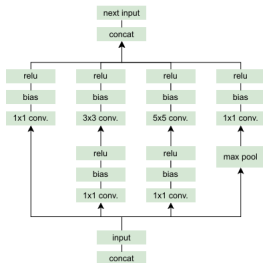
(c) Fusión horizontal de capas.

- **Dead layer removal**¹⁰
- **Vertical fusion:** Fusiona capas secuenciales con entradas similares (e.g., convolución, *bias*, ReLU) en una sola operación de kernel para reducir el número de lanzamientos.
- **Horizontal fusion:** Combina operaciones con parámetros similares sobre el mismo tensor de entrada en una única capa, reduciendo lanzamientos de kernel.
- **Quantization**¹¹
- **Automatic kernel tuning:** Selecciona los kernels más eficientes durante la compilación, optimizando la ejecución.

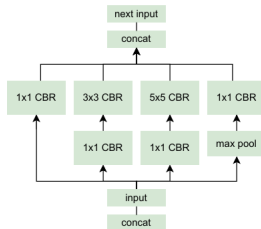
¹⁰Elimina capas cuyas salidas no son utilizadas (pesos con valores cercanos a cero), reduciendo el modelo sin afectar significativamente la salida.

¹¹Reduce la precisión del modelo (e.g., de fp32 a int8) para disminuir el uso de recursos, manteniendo precisión mediante calibración automática.

Vertical Fusion



(a) Red neuronal.



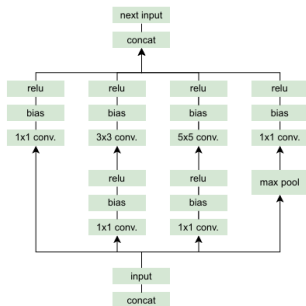
(b) Fusión vertical de capas.

- **Análisis de operaciones secuenciales:** En muchas arquitecturas de ANNs, las capas se organizan de manera secuencial. Una secuencia típica podría ser: **convolución** → **sesgo (bias)** → **activación ReLU**. En lugar de ejecutar cada operación como un kernel separado, TensorRT identifica estos patrones y los combina en una única operación¹².
- **Reducción del *overhead* de ejecución:** Cada lanzamiento de kernel desde el host hacia el dispositivo (GPU) incurre en un costo adicional de configuración, sincronización y comunicación¹³. Al fusionar operaciones en un único kernel, se reduce considerablemente este *overhead*, lo que se traduce en una latencia menor y un mayor rendimiento.

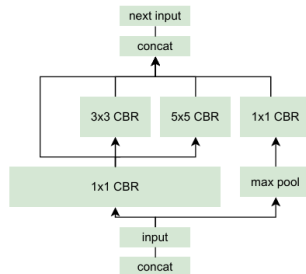
¹²Esto se logra siempre que las dimensiones y características de entrada/salida sean compatibles, permitiendo que un solo kernel realice múltiples operaciones de manera más eficiente.

¹³Aunque este costo pueda parecer insignificante, se acumula en redes profundas con muchas capas.

Horizontal Fusion



(a) Red neuronal.

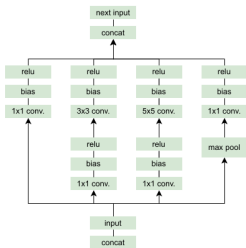


(b) Fusión horizontal de capas.

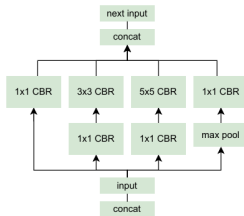
- **Combinación de capas paralelas con entradas comunes:** La fusión horizontal se aplica cuando múltiples capas reciben el mismo tensor de entrada y realizan operaciones equivalentes (por ejemplo, convoluciones con filtros 1×1 pero con diferentes pesos).
- En lugar de lanzar múltiples kernels, uno por cada capa, TensorRT agrupa estas operaciones en un solo kernel. Esto reduce el número de lanzamientos y mejora la eficiencia de cómputo.
- Tras la operación unificada, los resultados deben **separarse** para mantener la estructura original del grafo, de modo que cada rama posterior reciba su correspondiente salida.

Este proceso se ilustra en la Figura, donde tres capas CBR paralelas son fusionadas en una única operación.

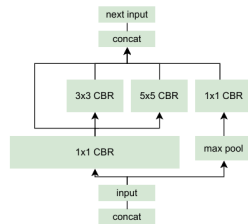
Automatic kernel tuning



(a) Red neuronal.



(b) Fusión vertical de capas.



(c) Fusión horizontal de capas.

- **Exploración y selección de kernels óptimos:** Durante esta etapa, el *builder* de TensorRT selecciona los kernels más adecuados para ejecutar cada operación del modelo, basándose en el nivel de optimización definido mediante el parámetro *builder optimization level*.
- **TensorRT posee una biblioteca de kernels especializados para diferentes tipos de capas y configuraciones, y el proceso de *tuning* consiste en evaluar iterativamente cuál kernel ofrece el mejor rendimiento en el hardware objetivo¹⁴.**
- **Naturaleza no determinística:** El proceso no es completamente determinista: optimizar el mismo modelo varias veces podría generar *engines* de inferencia distintos¹⁵.

¹⁴Esta evaluación toma en cuenta factores como el tamaño del *batch*, las dimensiones del filtro y la arquitectura de la GPU.

¹⁵Dado que la medición del tiempo de ejecución se realiza en condiciones reales, el *tuning* puede verse afectado por factores externos como procesos en segundo plano o variaciones en la frecuencia de la GPU.

Running Phase



Fase de ejecución del motor de inferencia optimizado.

- **Serialize engine:** guardar el modelo optimizado de TensorRT en un archivo binario (.plan file).
- **Deserialize engine:** cargar ese archivo y reconstruir el motor en memoria para ejecutar inferencia.

- ▶ **Deserialización del plan optimizado:** Se toma el plan generado durante la fase de optimización y se **deserializa para evitar optimizaciones en tiempo de ejecución**¹⁶, permitiendo así una ejecución más eficiente.
- ▶ **Generación del inference engine:** A partir del plan deserializado, se construye el ***inference engine***, encargado de ejecutar el modelo de forma optimizada en el hardware objetivo.
- ▶ **Creación del contexto de ejecución:** Contiene toda la información necesaria para llevar a cabo inferencias, incluyendo referencias a la memoria del dispositivo, configuraciones de entrada y salida.
- ▶ **Ejecución de inferencias:** Una vez definido el contexto de ejecución, se especifican las entradas y se obtienen las salidas correspondientes. En plataformas CUDA, este contexto representa el entorno donde se ejecutan los kernels y se gestiona la memoria del dispositivo.

¹⁶Se carga el plan previamente optimizado y guardado para evitar tener que hacer toda la optimización otra vez, lo cual ahorra tiempo y recursos. Al deserializarlo, se recupera el engine listo para hacer inferencias sin necesidad de volver a compilar o afinar el modelo.

- ▶ Las plataformas Jetson permiten configurar el **modo de consumo energético**, el cual:
 - Controla el número de núcleos activos.
 - Establece un límite de consumo energético permitido.
- ▶ Estas configuraciones son independientes de las usadas por TensorRT.
- ▶ Se usaron los modos activos al iniciar el dispositivo (según configuración de fábrica), detallados en la siguiente tabla.
- ▶ Nota: Estos modos no necesariamente coinciden con los predeterminados de la documentación oficial¹⁷.

Plataforma	Modo de Consumo	Power budget [W]	Online CPU
Jetson Xavier NX	Modo PM5	10	4
Jetson Xavier AGX	Modo PM7	15	4
Jetson Orin Nano	Modo PM0	15	6
Jetson Orin AGX	Modo PM2	30	8

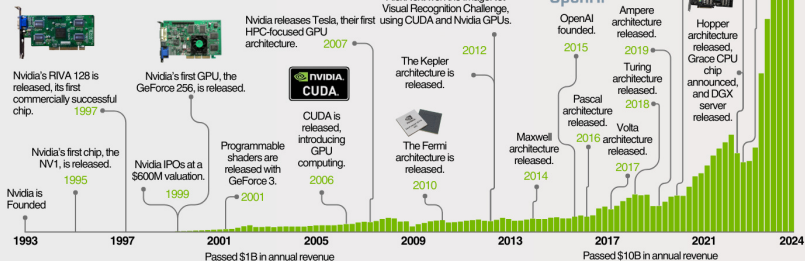
¹⁷<https://docs.nvidia.com/jetson/archives/r35.1/DeveloperGuide/text/SD/PlatformPowerAndPerformance/JetsonOrinNxSeriesAndJetsonAgxOrinSeries.html#supported-modes-and-power-efficiency>

GPU Architectures

The Rise of Nvidia

The key moments leading to its rise from a market cap of \$600M to \$3.5T.

Nvidia's Quarterly Revenue (since IPO)



Jetson Orin Nano Super Developer Kit

AI Performance	67 INT8 TOPS
GPU	NVIDIA Ampere architecture with 1024 CUDA cores and 32 tensor cores
CPU	6-core Arm® Cortex®-A78AE v8.2 64-bit CPU 1.5MB L2 + 4MB L3
Memory	8GB 128-bit LPDDR5 102 GB/s
Storage	Supports SD card slot and external NVMe
Power	7W-25W

* Refer to the Software Features section of the latest NVIDIA Jetson Linux Developer Guide for a list of supported features.

[Datasheet - NVIDIA Jetson Orin Nano Super Developer Kit](#)

View Jetson Orin Technical Specifications

	Jetson AGX Orin series				Jetson Orin NX series		Jetson Orin Nano series		
	Jetson AGX Orin Developer Kit	Jetson AGX Orin 64GB	Jetson AGX Orin Industrial	Jetson AGX Orin 32GB	Jetson Orin NX 16GB	Jetson Orin NX 8GB	Jetson Orin Nano Super Developer Kit	Jetson Orin Nano 8GB	Jetson Orin Nano 4GB
AI Performance	275 TOPS		248 TOPs	200 TOPS	157 TOPS	117 TOPS	67 TOPS	67 TOPS	34 TOPS
GPU	2048-core NVIDIA Ampere architecture GPU with 64 Tensor Cores			1792-core NVIDIA Ampere c GPU with 56 Tensor Cores	1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores		1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores		512-core NVIDIA Ampere architecture GPU with 16 Tensor Cores
GPU Max Frequency	1.3 GHz		1.2 GHz	930 MHz	1173MHz	1173MHz	1020MHz	1020MHz	1020MHz

View Jetson Orin Technical Specifications
 Getting Started with Jetson AGX Orin Developer Kit
 NVIDIA Ampere Architecture In-Depth

Multi-instance GPU (MIG) virtualization and GPU partitioning capability

- ▶ Para la GPU NVIDIA A100, MIG permite **dividir la GPU NVIDIA A100 en hasta siete instancias independientes para aplicaciones CUDA.**
- ▶ Cada instancia tiene procesadores, caminos de memoria, puertos de crossbar, bancos de caché L2, controladores de memoria y buses DRAM asignados de **forma exclusiva.**
- ▶ Cada usuario mantiene la misma asignación de caché L2 y ancho de banda de DRAM, incluso si otros saturan sus recursos.
- ▶ MIG permite un **mayor aprovechamiento del hardware** de la GPU, y garantiza **aislamiento** entre clientes.
- ▶ Es especialmente útil en **entornos multi-tenant de *Cloud Service Providers***, donde se requiere:
 - Aislamiento entre máquinas virtuales, contenedores y procesos.
 - Garantías de utilización de GPU para distintos clientes.

NVIDIA GA100 / A100 GPU Architecture

► GA100 full GPU:

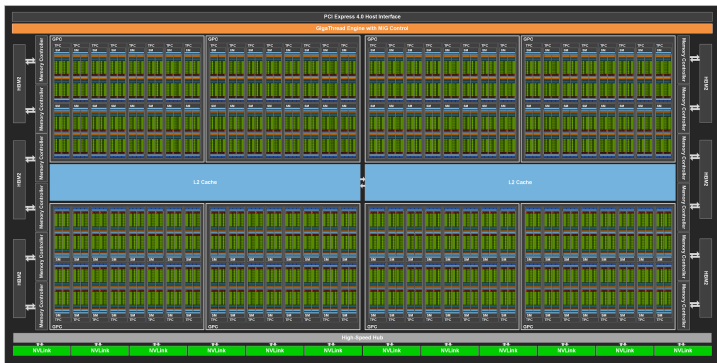
- 8 GPCs, 8 TPCs/GPC, 2 SMs/TPC
- 128 SMs (64 FP32 CUDA cores/SM)
- Total: 8192 FP32 CUDA Cores
- 512 Tensor Cores (3rd gen)
- 6 HBM2 stacks, 12×512-bit memory controllers

► A100 implementation (based on GA100):

- 7 GPCs, hasta 16 SMs/GPC
- 108 SMs en total
- 6912 FP32 CUDA Cores
- 432 Tensor Cores (3rd gen)
- 5 HBM2 stacks, 10×512-bit memory controllers



- ▶ **TPC (Texture Processing Cluster):** Subdivisión dentro del GPC. Contiene 2 SMs y unidades de texturas/cómputo.
- ▶ **GPC (Graphics Processing Cluster):** Agrupa TPCs. Controla y distribuye trabajo en la GPU.
- ▶ **SM (Streaming Multiprocessor):** Ejecuta los hilos CUDA en paralelo. Incluye:
 - **Núcleos FP32 CUDA:** operaciones en coma flotante simple.
 - **Tensor Cores:** especializados en operaciones de matrices (IA, DL).
- ▶ **HBM2 (High Bandwidth Memory):** Memoria de alta velocidad conectada directamente al chip (GB/s de ancho de banda)
- ▶ **Memory Controllers:** Interfases entre los SMs/L2 Cache y la memoria HBM2. Cada uno maneja un canal de memoria de 512 bits.
- ▶ **NVLink:** Interconexión de alta velocidad entre GPUs.
- ▶ **PCIe 4.0 Host Interface:** Conexión CPU/host.
- ▶ **L2 Cache:** Caché compartida entre todos los SMs, reduce accesos a memoria global.



How the GA100/A100 Architecture Works

► Jerarquía de procesamiento

- La GPU se organiza en **GPCs**.
- Cada GPC contiene **TPCs**.
- Cada TPC incluye **SMs**

► SM

- Ejecuta miles de hilos en paralelo organizados en **warps** (32 hilos).
- Contiene **CUDA Cores** (operaciones FP32) y **Tensor Cores** (operaciones matriciales para IA).

► Memoria

- **L2 Cache** compartida conecta todos los SMs.
- **HBM2 (High Bandwidth Memory)** provee gran ancho de banda para datos.

► Interconexiones

- **PCIe 4.0** conecta la GPU al CPU/host.
- **NVLink** enlaza múltiples GPUs a alta velocidad.

► MIG

- Permite dividir la A100 en hasta 7 GPUs lógicas independientes.

Performance Assessment

- ▶ Existen dos métricas principales para caracterizar el rendimiento de un sistema digital en términos de "*speed*".
 - **Latency (response time or execution time)**¹⁸¹⁹
 - Cuánto tiempo se tarda en generar un resultado a partir de un conjunto de entrada.
 - Cuanto tarda en completar una tarea.
 - **Throughput (bandwidth)**
 - Número de resultados producidos por unidad de tiempo.
 - Típicamente relevante para datacenters-

¹⁸Medida importante para servicios basados en IA (Alexa y Siri)

¹⁹Aplicaciones *safe-critical*

- En general, nuestro objetivo es mejorar el rendimiento de una computadora A reduciendo el tiempo de ejecución.

$$Performance_A = \frac{1}{Execution\ time_A}$$

- Cuando decimos: "el sistema A es n veces más rápido que el sistema B "

$$\frac{Performance_A}{Performance_B} = \frac{Execution\ time_B}{Execution\ time_A} = n$$

- ▶ La latencia se define como el tiempo necesario para procesar una única entrada a través de la red con un *batch size* de inferencia igual a uno.
- ▶ Generalmente, se excluye el preprocesamiento de la entrada.
- ▶ En los experimentos, se mide usando rondas de *warm-up*²⁰ equivalentes al 10 % del total de datos del conjunto de validación.
- ▶ El *warm-up* permite estabilizar el sistema antes de la medición, evitando valores atípicos debido a la carga inicial del modelo.

²⁰pruebas iniciales o enviar tráfico de prueba antes de comenzar las mediciones de latencia reales

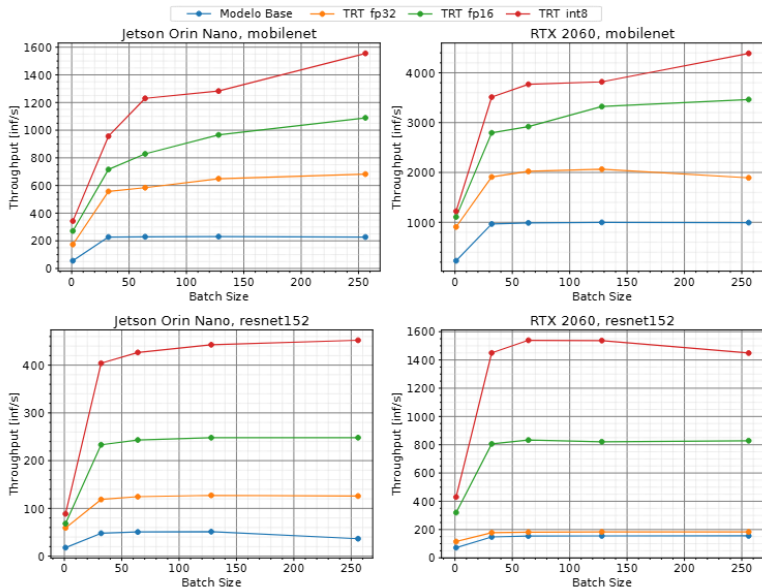
- ▶ La latencia se mide calculando el tiempo total de ejecución de cada inferencia.
- ▶ Se incluyen transferencias de datos entre la CPU y la GPU, y viceversa.
 - *Overhead* asociado con la gestión de la memoria.
- ▶ En python, se puede utilizar la función `time` del módulo `time` de Python.
- ▶ Se registran valores de latencia para una única ejecución en todo el conjunto de validación.
- ▶ Se calcula la latencia promedio y la latencia máxima.

- ▶ El *throughput* indica la cantidad de datos procesados por unidad de tiempo.
- ▶ Se puede calcular dividiendo el número de entradas del modelo (*batch size* de inferencia) por el tiempo necesario para procesarlas.
- ▶ El tiempo de procesamiento se mide de la misma manera que la latencia, usando un *warm-up*²¹ del 10 %.
- ▶ Se incluyen los tiempos de transferencia de datos entre la CPU y la GPU.

²¹Unas inferencias iniciales antes de las mediciones

- ▶ Se registran los tiempos de procesamiento de cada lote de entradas con un tamaño de *batch size* específico.
- ▶ Se calcula un tiempo de procesamiento promedio para determinar el *throughput* promedio.
- ▶ Una recomendación para evaluar *throughput*, es evaluar distintos tamaños de *batch size*, comenzando con 1, luego 32 y duplicándolo hasta 256.
- ▶ Aumentar el *batch size* puede incrementar el *throughput*.

Medición de Throughput



22 Comparación del throughput en función del batch size de inferencia para los modelos MobileNet y ResNet152 en las plataformas Jetson Orin Nano y RTX 2060.

- **Latencia** (tiempo por inferencia):

$$\text{Latencia} = \frac{T_{\text{ejecución total}}}{N_{\text{inferencias}}}$$

- **Throughput** (inferencias por segundo):

$$\text{Throughput} = \frac{\text{Batch Size}}{T_{\text{proceso}}}$$

- **Speedup** (ganancia respecto a baseline):

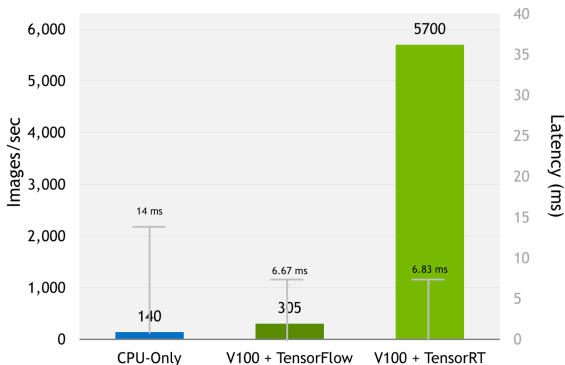
$$\text{Speedup} = \frac{T_{\text{baseline}}}{T_{\text{TensorRT}}}$$

- **Eficiencia Energética:**

$$\text{Eficiencia} = \frac{\text{Throughput}}{\text{Potencia (W)}}$$

Estas métricas permiten comparar configuraciones fp32, fp16 e int8 en distintas GPUs.

Up to 40x Faster CNNs on V100 vs. CPU-Only Under 7ms Latency (ResNet50)

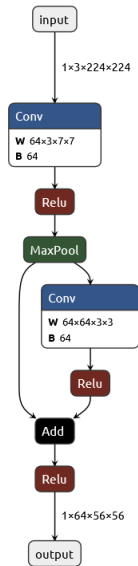


Inference throughput (images/sec) on ResNet50. **V100 + TensorRT**: NVIDIA TensorRT (FP16), batch size 39, Tesla V100-SXM2-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On **V100 + TensorFlow**: Preview of volta optimized TensorFlow (FP16), batch size 2, Tesla V100-PCIE-16GB, E5-2690 v4@2.60GHz 3.5GHz Turbo (Broadwell) HT On. **CPU-Only**: Intel Xeon-D 1587 Broadwell-E CPU and Intel DL SDK. Score doubled to comprehend Intel's stated claim of 2x performance improvement on Skylake with AVX512.

- ▶ El tamaño del modelo y la capacidad de la red entre procesadores influyen en el rendimiento.
- ▶ Hay varios factores en el aprendizaje profundo, como el número de capas, el número de neuronas por capa, la complejidad del cálculo por capa y el número de conexiones entre un nodo de una capa y los nodos de las capas siguientes.

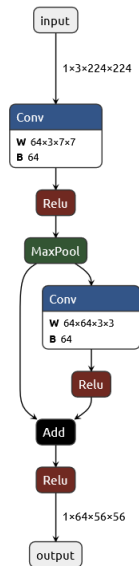
Graph Format en Machine Learning

- Un modelo de Machine Learning se representa como un **grafo computacional**, facilitando su interoperabilidad entre frameworks.
- Permite visualizar cómo los datos se procesan a través de las capas del modelo.



Elementos del Grafo Computacional

- **Nodos:** Representan capas u operaciones del modelo. Ejemplo: convolucional (azul).
- **Aristas:** Simbolizan los tensores y muestran el flujo de datos entre nodos.
- **Parámetros:** Son los pesos (W) y sesgos (B), ajustados en el entrenamiento.



- ▶ El tamaño del modelo y el ancho de banda de la red entre procesadores afectan el rendimiento²⁴.
- ▶ Factores clave en el aprendizaje profundo:
 - **Número de capas:** Más capas aumentan la complejidad del modelo.
 - **Neuronas por capa:** Determina la capacidad de representación.
 - **Complejidad computacional por capa:** Afecta el tiempo de entrenamiento e inferencia.
 - **Conexiones entre capas:** Más conexiones implican un mayor costo computacional.

Optimizar el tamaño y la arquitectura del modelo es crucial para equilibrar precisión y eficiencia.

²⁴En sistemas con PCIe, NVLink, o Ethernet, un ancho de banda bajo puede causar cuellos de botella.

Tamaño del Modelo (Model Size)

- ▶ Se refiere al tamaño total ocupado por los parámetros y la estructura del modelo.
- ▶ Se mide en términos de la cantidad de memoria necesaria para almacenarlo (en megabytes).
- ▶ La medición se puede realizar utilizando la función `getsize` del módulo `os` de Python.
 - Métodos como cuantización, *pruning* y *knowledge distillation* pueden reducir significativamente el tamaño del modelo.
- ▶ Modelos almacenados en `fp32` ocuparán más espacio que en `fp16` o `int8` (cuantización).

► Classification closeness

- Evalúa el impacto de las optimizaciones de TensorRT sobre las salidas numéricas de las redes neuronales.

Se utiliza `torch.isclose` para comparar las salidas:

$$|a - b| \leq \text{atol} + \text{rtol} \cdot |b|$$

- Se fija `rtol = 0`. Los valores de `atol` se definen como porcentajes del valor máximo dentro del percentil 90 de las salidas del modelo base.

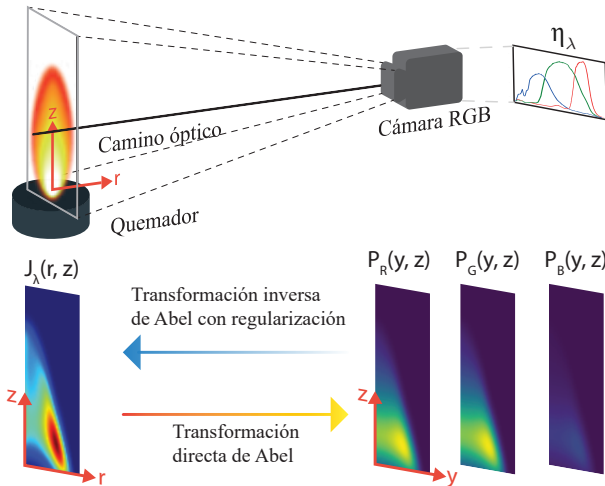
► Uso de memoria durante la inferencia

- Evalúa el consumo de RAM durante todo el proceso de inferencia: carga, preprocesamiento, inferencia y postprocesamiento.
- Se emplea `tegrastat` con muestreo de 1 ms para registrar el uso de memoria.
- Se calcula el uso promedio y máximo de memoria para distintas configuraciones y valores de batch size.

Ejemplo de aplicación

Plano de simetría 2D de la
llama 3D CLAD

Proyección 2D de la llama 3D
captada por la cámara



Arquitectura comúnmente empleada para resolver problemas inversos en imágenes y de deconvolución.

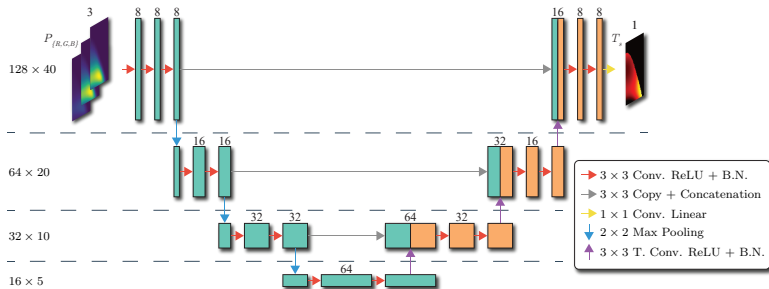


Figura: Arquitectura U-Net²⁵.

²⁵Ronneberger et al., U-Net: Convolutional Networks for Biomedical Image Segmentation, in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, Cham: Springer International Publishing, 2015, pp. 234-241.

Attention U-Net

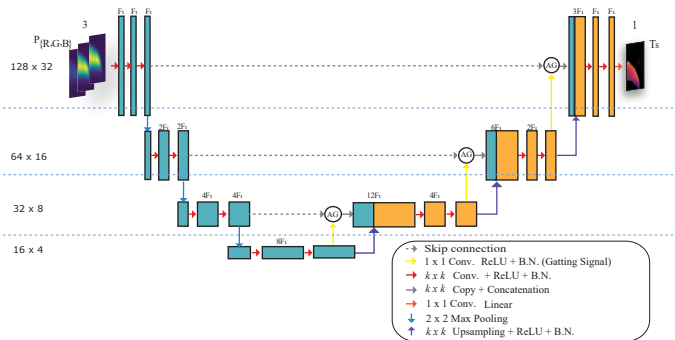


Figura: Arquitectura Attention U-Net.

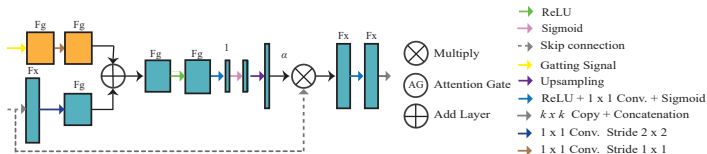


Figura: Compuerta de atención^a.

^aOktay et al., .Attention U-Net: Learning Where to Look for the Pancreas.,arXiv preprint arXiv:1804.03999, 2018.

El flujo de trabajo utilizado para obtener los *inference engines* derivados de los modelos base U-Net y Attention U-Net sigue el esquema PyTorch-ONNX-TensorRT.

Plataformas de hardware seleccionadas: una plataforma embebida Jetson Orin AGX y un computador de escritorio equipado con una tarjeta gráfica RTX 3060.

Códigos: <https://github.com/Juanx65/UNetTRT>

Tabla 1: Propiedades de los modelos U-Net y Attention (**Att.**) U-Net en sus distintas versiones: modelo base, TRT fp32, TRT fp16 y TRT int8.

Hardware	Modelo	Propiedades		
		Tamaño [MB]	# Capas	# Parámetros
RTX 3060	U-Net Base	7.25	91	1884631
	U-Net TRT fp32	10.75	23	1880040
	U-Net TRT fp16	4.45	25	1880040
	U-Net TRT int8	2.84	31	1880040
	Att. U-Net Base	33.75	114	8828070
	Att. U-Net TRT fp32	35.13	70	8823221
	Att. U-Net TRT fp16	18.57	46	8823221
	Att. U-Net TRT int8	11.06	48	8823221
Jetson Orin AGX	U-Net Base	7.25	91	1884631
	U-Net TRT fp32	7.88	24	1880040
	U-Net TRT fp16	4.66	25	1880040
	U-Net TRT int8	2.64	30	1880040
	Att. U-Net Base	33.75	114	8828070
	Att. U-Net TRT fp32	35.33	61	8823221
	Att. U-Net TRT fp16	18.39	46	8823221
	Att. U-Net TRT int8	9.75	30	8823221

Tabla 2: Resultados de **regression accuracy** en Kelvin (K) bajo diferentes condiciones de llama.

	Modelo	Cond. de llama	Modelo base	TRT fp32	TRT fp16	TRT int8
RTX 3060	U-Net	Yale-32	15.3	19.4	19.5	21.3
		Yale-40	40.8	39.8	39.6	36.7
		Yale-60	31.7	35.4	35.4	35.2
	Att. U-Net	Yale-32	31.0	31.0	31.0	29.9
		Yale-40	38.4	38.4	38.4	56.6
		Yale-60	28.4	28.4	28.4	41.6
J. Orin AGX	U-Net	Yale-32	15.3	19.4	19.5	19.6
		Yale-40	40.8	39.8	39.6	37.9
		Yale-60	31.7	35.4	35.4	33.9
	Att. U-Net	Yale-32	31.0	31.0	31.0	29.4
		Yale-40	38.4	38.4	38.4	49.5
		Yale-60	28.4	28.4	28.4	43.2

Soot Pyrometry

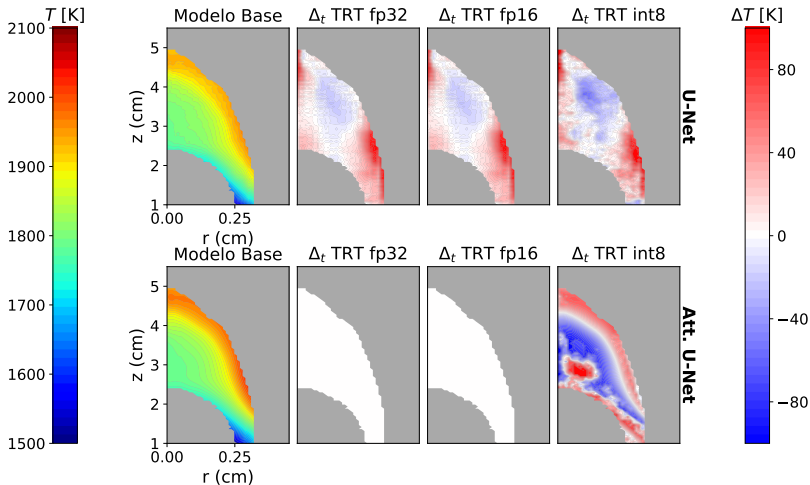


Figura 2: Comparación de la predicción de T_s entre el modelo base y los modelos optimizados bajo la condición de llama Yale-60. Se incluye el mapa de error ΔT respecto al modelo base.

Para las configuraciones fp32 y fp16, el modelo Attention U-Net alcanza un closeness del 100 % con una tolerancia del 0.5 %. En contraste, la red U-Net, bajo la misma condición de atol1, solo alcanza un closeness del 2 %, lo que evidencia una mayor robustez de Attention U-Net frente a la optimización con cuantización fp32 y fp16.

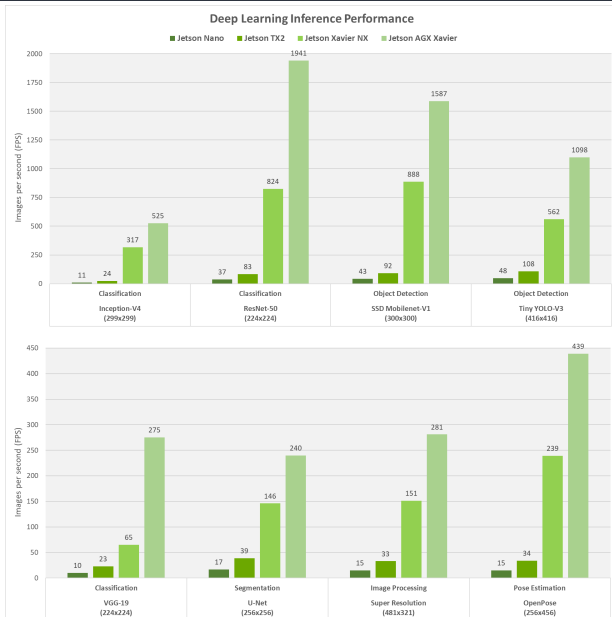
Tabla 3: Resultados de closeness (%) según diferentes razones de tolerancia absoluta para los modelos U-Net y Attention U-Net.

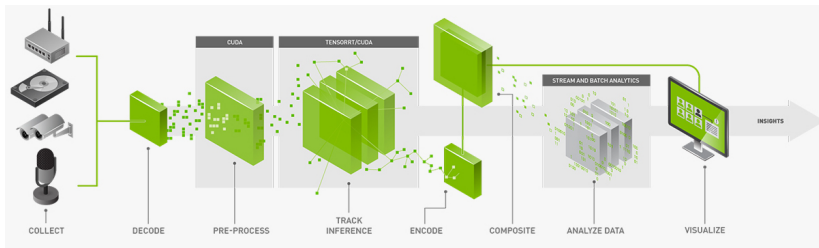
			atol 0.005	atol 0.01	atol 0.1	atol 0.2	atol 0.5	atol 1
RTX 3060	U-Net	fp32	3.10 %	6.51 %	61.28 %	85.52 %	97.94 %	99.99 %
		fp16	3.10 %	6.43 %	61.36 %	85.51 %	97.94 %	99.99 %
		int8	3.46 %	6.92 %	62.53 %	85.27 %	97.08 %	99.98 %
	Att. U-Net	fp32	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		fp16	99.97 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		int8	4.53 %	8.94 %	73.67 %	93.21 %	99.87 %	100.00 %
J.O.AGX	U-Net	fp32	3.14 %	6.47 %	61.81 %	85.63 %	98.00 %	99.99 %
		fp16	3.16 %	6.47 %	61.82 %	85.62 %	98.00 %	99.99 %
		int8	3.11 %	5.65 %	51.87 %	82.60 %	98.62 %	99.97 %
	Att. U-Net	fp32	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		fp16	99.97 %	100.00 %	100.00 %	100.00 %	100.00 %	100.00 %
		int8	2.55 %	4.99 %	37.79 %	59.58 %	91.32 %	99.96 %

Tabla 4: Resultados de latencia para un batch size de inferencia de 1 y de throughput para batch sizes de inferencia entre 2 y 32 en la plataforma Jetson Orin AGX.

	Modelo	Latencia [ms]		Throughput [inf/s]				
		ave.	max.	2	4	8	16	32
U-Net	Modelo Base	126.5	714.5	27.17	31.29	81.33	117.49	208.05
	TRT fp16	112.2	139.7	35.18	52.53	125.72	135.41	268.36
Att. U-Net	Modelo Base	134.2	753.8	15.20	30.97	58.53	92.60	122.94
	TRT fp16	115.4	312.8	19.74	42.10	65.11	123.34	220.30

Benchmarking





- ▶ **Qué es DeepStream SDK?** Plataforma de NVIDIA para construir aplicaciones de video en tiempo real, potenciadas con IA, en edge y cloud.
 - Optimizado para procesamiento multi-stream y baja latencia usando CUDA y TensorRT.
 - Soporta fuentes heterogéneas (cámaras, sensores, medios grabados).
- ▶ **Arquitectura del pipeline** Etapas modulares.
- ▶ **Integración con GStreamer**
 - Basado en el framework GStreamer.
 - Plugins para decoding, preprocessing, AI inference y analytics.

► GPU Zero-Copy Buffers

- Evita transferencias innecesarias CPUGPU usando memoria compartida en GPU.
- Reduce la latencia y el overhead de copia de datos.

► Pipelines Asíncronos

- Uso de *asynchronous pipelines* en GStreamer para superponer etapas (decode, inference, encode).
- Aprovecha CUDA streams para ejecutar kernels en paralelo y optimizar el throughput.

► TensorRT Optimization

- Conversión y optimización de modelos DNN para ejecución en GPU (FP32, FP16, INT8).
- *Layer fusion*, optimización de memoria y scheduling eficiente.

► Batching Dinámico

- Procesa múltiples frames de diferentes streams en un solo kernel launch.
- Minimiza overhead y maximiza la utilización del GPU.

► Pipeline optimizado para *low-latency, high-throughput video analytics* en escenarios edge y cloud.

► Getting Started with AI on Jetson Nano

- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-RX-02+V2

► Building Video AI Applications at the Edge on Jetson Nano

- https://learn.nvidia.com/courses/course-detail?course_id=course-v1:DLI+S-IV-02+V2



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

TensorRT SDK NVIDIA

Sección 1 — Fundamentos y Arquitectura

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile